

Syntropy Memory Engine

Como a Syntropy consolida e reutiliza experiência operacional.

Aprender e lembrar não são a mesma coisa. A Learning Engine define como a Syntropy evolui. A Memory Engine define **o que ela retém, por quanto tempo, como esquece, e como transforma dados brutos em sabedoria reutilizável.**

Por que Memória é Diferente de Aprendizado

Aprendizado	Memória
Detecta padrões	Armazena padrões detectados
Acontece em tempo real	Persiste após o evento
Efêmero (durante análise)	Durável (meses/anos)
“Eu percebi algo”	“Eu lembro que isso já aconteceu”

Sem memória, a Syntropy seria inteligente mas amnésica — descobriria os mesmos padrões repetidamente sem acumular experiência operacional.

Ciclo de Vida da Memória

1. CAPTURA – Dados brutos durante o evento Tudo que acontece é registrado Volume: alto (milhares de registros por evento) Retenção: permanente (banco de dados)
2. CONSOLIDAÇÃO – Após o evento terminar Dados brutos → Resumo executivo + métricas-chave Volume: reduzido (1 relatório por evento) Retenção: permanente
3. ABSTRAÇÃO – Após múltiplos eventos Resumos → Perfil do produtor + tendências Volume: mínimo (1 perfil por produtor) Retenção: permanente, atualizado continuamente
4. SABEDORIA – Após escala da rede Perfis → Benchmark + melhores práticas Volume: agregado (1 por segmento de mercado) Retenção: permanente, anonimizado

O que a Syntropy Lembra

Memória de Curto Prazo (durante o evento)

O que	Duração	Uso
Snapshot atual	Substituído a cada ciclo	Decisão em tempo real
Baselines (30min)	Janela deslizante	Deteção de anomalias
Sugestões pendentes	Até aprovação/rejeição	Evitar duplicatas
Ações executadas	Até fim do evento	Evitar repetição

Memória de Médio Prazo (entre eventos do mesmo produtor)

O que	Duração	Uso
Relatório pós-evento (Vision)	Permanente	Comparação com próximo evento
Health Score	Permanente	Tendência do produtor
Top produtos	Permanente	Sugestão de cardápio
Produtos que encailharam	Permanente	Alerta preventivo
Performance por bar	Permanente	Layout operacional
Horário de pico	Permanente	Escala de equipe
Mix de pagamento	Permanente	Planejamento financeiro
Sugestões aceitas/rejeitadas	Permanente	Calibração da IA

Memória de Longo Prazo (rede)

O que	Duração	Uso
Benchmark por segmento	Permanente	Comparação entre produtores
Sazonalidade	Permanente	Previsão de demanda
Correlação clima × consumo	Permanente	Ajuste de estoque
Melhores práticas	Permanente	Recomendações

Como a Syntropy Esquece

Nem tudo deve ser lembrado para sempre. A Syntropy tem regras de **esquecimento intencional**:

Regras de Decay (Decaimento)

DADOS BRUTOS (pedidos, transações individuais)

→ Mantém: 12 meses (compliance + análise)

→ Depois: Agrega em métricas, apaga granularidade

SUGESTÕES IGNORADAS

→ Mantém: 3 meses

→ Depois: Conta como "tipo X tem baixa aceitação"

DADOS DE PARTICIPANTE (LGPD)

→ Mantém: Até solicitação de exclusão

→ Depois: Anonimiza (mantém estatística, remove PII)

EVENTOS MUITO ANTIGOS (> 2 anos)

→ Mantém: Resumo + métricas-chave

→ Depois: Dados granulares podem ser arquivados

Por que esquecer é importante

1. **Performance** — Menos dados = consultas mais rápidas
2. **Relevância** — Dados de 3 anos atrás podem ser enganosos (mercado mudou)
3. **LGPD** — Obrigação legal de não reter dados desnecessários
4. **Foco** — Memória seletiva é mais útil que memória total

Como a Syntropy Resume

O processo de transformar dados brutos em memória reutilizável:

Evento → Resumo (Consolidação)

847 pedidos
+ 2.341 itens
+ 156 transações Pix
+ 89 transações cartão
+ 12 alertas de estoque
+ 3 sugestões aceitas
+ 1 sugestão rejeitada

↓ CONSOLIDAÇÃO (Vision)

```
{  
  faturamento: R$ 63.500,  
  ticketMedio: R$ 75,  
  participantes: 420,  
  healthScore: 82,  
  topProduto: "Cerveja Premium",  
  picoHorario: "22:30-23:30",  
  mixPagamento: { pix: 64%, cartao: 36% },  
  recomendacoes: [  
    "Aumentar estoque de cerveja em 20%",  
    "Adicionar 1 operador no Bar 2 após 22h"  
  ]  
}
```

Múltiplos Resumos → Perfil (Abstração)

Evento 1: healthScore 72, ticket R\$ 65, 300 pessoas
Evento 2: healthScore 78, ticket R\$ 72, 380 pessoas
Evento 3: healthScore 82, ticket R\$ 75, 420 pessoas
Evento 4: healthScore 85, ticket R\$ 80, 450 pessoas

↓ ABSTRAÇÃO

```
{  
  tendencia: "crescente",  
  healthScoreMedio: 79,  
  ticketMedioHistorico: R$ 73,  
  publicoMedio: 388,  
  crescimentoMedio: "+12% por evento",  
  fortalezas: ["pricing", "cardápio"],  
  fragilidades: ["estoque", "escala de equipe"],  
  segmento: "festival_medio"  
}
```

Múltiplos Perfis → Benchmark (Sabedoria)

Produtor A (festival_medio): ticket R\$ 73, health 79
Produtor B (festival_medio): ticket R\$ 68, health 74
Produtor C (festival_medio): ticket R\$ 82, health 86
Produtor D (festival_medio): ticket R\$ 71, health 77
...

↓ SABEDORIA

```
{  
  segmento: "festival_medio",  
  ticketMedioMercado: R$ 74,  
  healthScoreMercado: 79,  
  melhorPratica: "Combos aumentam ticket em 15%",  
  alertaComum: "60% dos festivais subestimam cerveja",  
  topCategoria: "Drinks prontos crescendo 30% a.a."  
}
```

Estrutura de Memória (Implementação)

Onde cada tipo de memória vive

Tipo	Storage	Tabela/Estrutura
Dados brutos (pedidos)	MySQL	orders , order_items
Dados brutos (transações)	MySQL	orders (paymentMethod)
Dados brutos (estoque)	MySQL	ingredients , ingredient_movements
Insights gerados	MySQL	ai_insights
Ações executadas	MySQL	ia_actions
Relatório pós-evento	MySQL	post_event_reports
Perfil do produtor	✗ Não existe	producer_profiles (a criar)
Benchmark de rede	✗ Não existe	network_benchmarks (a criar)

Schema proposto para `producer_profiles`

```
CREATE TABLE producer_profiles (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  userId INT NOT NULL,  
  totalEvents INT DEFAULT 0,  
  avgHealthScore DECIMAL(5,2),  
  avgTicket DECIMAL(10,2),  
  avgAttendees INT,  
  growthTrend ENUM('growing', 'stable', 'declining'),  
  topProducts JSON,          -- [{name, avgSales}]  
  weakProducts JSON,         -- [{name, reason}]  
  strengths JSON,            -- ["pricing", "cardápio"]  
  weaknesses JSON,           -- ["estoque", "equipe"]  
  segment VARCHAR(50),       -- "festival_medio", "corporativo_grande"  
  lastEventAt DATETIME,  
  updatedAt DATETIME DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (userId) REFERENCES users(id)  
);
```

Recall — Como a Syntropy Recupera Memórias

Quando a Syntropy precisa de uma memória, ela segue esta hierarquia:

Preciso de informação sobre [\[X\]](#)

- └ Memória de curto prazo (snapshot/baseline)?
 - └ Se sim → Usar (mais recente e relevante)
- └ Memória de médio prazo (último evento)?
 - └ Se sim → Comparar com atual
- └ Memória de longo prazo (perfil do produtor)?
 - └ Se sim → Usar como referência
- └ Memória de rede (benchmark)?
 - └ Se sim → Usar como fallback

Exemplo: “Quanto estoque de cerveja preciso?”

- 1. Curto prazo: "Consumo atual: 60/hora" (se evento ativo)
- 2. Médio prazo: "Último evento: 55/hora" (se tem histórico)
- 3. Longo prazo: "Média do produtor: 52/hora" (se 3+ eventos)
- 4. Rede: "Média de festivais: 48/hora" (se tem benchmark)

- Syntropy usa a fonte mais específica disponível
- Se tem curto prazo, ignora o resto (dado mais preciso)
- Se é planejamento (pré-evento), usa médio/longo prazo

Conflito de Memórias

Quando memórias diferentes dizem coisas contraditórias:

Situação	Resolução
Curto prazo ≠ Médio prazo	Curto prazo vence (dado mais recente)
Último evento ≠ Perfil	Último evento vence (mais relevante)
Perfil ≠ Benchmark	Perfil vence (mais específico)
Dados ≠ Intuição do produtor	Dados informam, produtor decide

Evolução da Memória

Fase	Capacidade	Status
V1 (atual)	Dados brutos + Vision pós-evento	✅ Implementado
V2	Perfil do produtor consolidado	❌ Próximo passo
V3	Recall hierárquico no Planner	❌ Planejado
V4	Benchmark de rede	❌ Requer escala
V5	Esquecimento inteligente (decay)	❌ Futuro

Para Desenvolvedores

Próximos passos de implementação:

1. Criar tabela `producer_profiles` (schema acima)
2. Trigger pós-Vision que atualiza o perfil
3. Planner consultar perfil ao gerar estimativas
4. API de recall hierárquico (curto → médio → longo → rede)
5. Job de decay para dados > 12 meses (agregar + limpar)

Regra fundamental: A Syntropy nunca esquece uma lição — ela esquece os detalhes, mas retém a sabedoria.

Syntropy Context Engine

Como a Syntropy entende que o mesmo dado muda de significado.

32°C num festival sertanejo é diferente de 32°C num casamento. 500 pessoas num bar é lotação; 500 num festival é vazio. O Context Engine é o que impede a Syntropy de dar recomendações genéricas — ela entende **onde, quando, para quem e em que situação** cada dado existe.

O Problema que o Context Engine Resolve

Sem contexto, a Syntropy seria um sistema de regras burras:

Dado	Sem contexto	Com contexto
32°C	“Está quente”	“Festival outdoor + 32°C = +40% cerveja, -20% drinks quentes”
Ticket R\$ 45	“Abaixo da média”	“Para universitário é alto; para corporativo é baixo”
200 pedidos/h	“Normal”	“Para 500 pessoas é ótimo; para 3000 é preocupante”
Fila 8min	“Aceitável”	“Para bar de festival OK; para evento VIP é inaceitável”
Cerveja encalhando	“Promover”	“Se é evento fitness, é esperado — não promover”

Dimensões de Contexto

A Syntropy avalia cada dado através de 7 dimensões:

1. TIPO DE EVENTO

Festival / Corporativo / Casamento / Bar / Festa

→ Define expectativas de comportamento

2. PERFIL DO PÚBLICO

Jovem / Adulto / Misto / VIP / Popular

→ Define ticket esperado e preferências

3. AMBIENTE FÍSICO

Indoor / Outdoor / Coberto / Praia / Urbano

→ Define impacto do clima e logística

4. MOMENTO TEMPORAL

Início / Pico / Fim / Madrugada

→ Define o que é "normal" para aquele horário

5. ESCALA

Pequeno (<200) / Médio (200-1000) / Grande (1000+)

→ Define thresholds e proporções

6. HISTÓRICO DO PRODUTOR

1º evento / Experiente / Padrão estabelecido

→ Define nível de confiança nas previsões

7. CONDIÇÕES EXTERNAS

Clima / Dia da semana / Feriado / Concorrência

→ Define fatores que afetam demanda

Como o Contexto Modifica Decisões

Exemplo 1: Temperatura alta

DADO: Temperatura = 34°C

CONTEXTO A: Festival sertanejo, outdoor, 2000 pessoas

- Impacto: +40% cerveja, +30% água, -50% drinks quentes
- Ação: Reforçar estoque de gelados, alertar equipe

CONTEXTO B: Evento corporativo, indoor com ar-condicionado

- Impacto: Nenhum (ambiente controlado)
- Ação: Nenhuma

CONTEXTO C: Casamento ao ar livre, 150 convidados

- Impacto: +20% espumante/drinks leves, preocupação com conforto
- Ação: Sugerir drinks refrescantes, alertar sobre hidratação

Exemplo 2: Fila de 10 minutos

DADO: Tempo médio de espera = 10 minutos

CONTEXTO A: Festival com 3000 pessoas, 5 bares

- Avaliação: NORMAL (público espera até 15min em festival)
- Ação: Monitorar, sem alerta

CONTEXTO B: Evento corporativo, 200 pessoas, 2 bares

- Avaliação: CRÍTICO (executivos não esperam)
- Ação: Alerta imediato, sugerir realocação

CONTEXTO C: Bar VIP exclusivo, 50 pessoas

- Avaliação: INACEITÁVEL (VIP = atendimento imediato)
- Ação: Alerta urgente, priorizar atendimento

Exemplo 3: Produto encalhado

DADO: Gin Tônica vendeu 4 unidades em 3 horas

CONTEXTO A: Festival universitário, público 18-25

- Avaliação: ESPERADO (público prefere cerveja/drinks baratos)
- Ação: Não promover, reduzir estoque no próximo

CONTEXTO B: Evento corporativo premium

- Avaliação: ANÔMALO (público deveria consumir drinks premium)
- Ação: Verificar preço, posição no cardápio, visibilidade

CONTEXTO C: Festival gastronômico

- Avaliação: OPORTUNIDADE (público aprecia, talvez não viu)
- Ação: Sugerir combo com prato, destacar no cardápio

Matriz de Contexto

A Syntropy cruza tipo de evento × dimensão para ajustar seus parâmetros:

Parâmetro	Festival	Corporativo	Casamento	Bar/Balada
Fila aceitável	12min	5min	3min	8min
Ticket esperado	R\$ 40-80	R\$ 80-150	R\$ 60-120	R\$ 50-100
Cerveja % mix	50-70%	20-40%	15-30%	40-60%
Pico esperado	22h-00h	19h-21h	21h-23h	23h-02h
Tolerância a ruptura	Alta	Baixa	Muito baixa	Média
Sensibilidade a preço	Alta	Baixa	Média	Alta

Como o Contexto é Determinado

Fontes de contexto (em ordem de confiança)

1. Configuração explícita **do produtor** (tipo, público, local)
 - Confiança: ALTA
 - Quando: Setup **do** evento
2. Dados **do onboarding** (Syntropy Planner)
 - Confiança: ALTA
 - Quando: Conversa com produtor
3. Inferência dos **dados** (ticket, horários, produtos)
 - Confiança: MÉDIA
 - Quando: Durante o evento
4. Histórico **do produtor** (eventos anteriores)
 - Confiança: MÉDIA-ALTA
 - Quando: **2º** evento em diante
5. Benchmark de **rede** (eventos similares)
 - Confiança: MÉDIA-BAIXA
 - Quando: Sem histórico próprio

Resolução de ambiguidade

Se contexto explícito existe → Usar
Se não → Inferir dos dados
Se dados insuficientes → Usar **default do** segmento
Se nenhum segmento → Usar **default** genérico (festival médio)

Contexto Dinâmico (Muda Durante o Evento)

O contexto não é estático. Ele pode mudar:

Mudança	Detecção	Ajuste
Chuva começa	API meteorológica	Reduzir projeção de público
Público maior que esperado	Contagem de pedidos	Aumentar thresholds
Horário avançou para madrugada	Relógio	Ajustar “normal” para horário
Bar fechou	Status do ponto de venda	Redistribuir carga
Atração principal começou	Pico súbito de pedidos	Não tratar como anomalia

Exemplo: Pico por atração

SITUAÇÃO: Pedidos sobem 200% em 5 minutos

SEM CONTEXTO: "Anomalia! Possível fraude!"

COM CONTEXTO:

- Evento tem grade de atrações
- Atração principal às 23h
- São 23:05

→ "Pico esperado por início da atração principal"

→ Ação: Não alertar fraude, mas alertar equipe para reforço

Implementação Atual vs Planejada

Capacidade	Status	Onde
Tipo de evento no setup	✓	<code>events.type</code>
Público esperado	✓	<code>events.expectedAttendees</code>
Local (indoor/outdoor)	◆ Parcial	Onboarding, não estruturado
Thresholds por tipo	✗	Hardcoded genérico
Contexto dinâmico (clima)	✗	Apenas pós-evento
Contexto dinâmico (horário)	◆	Baseline adapta, mas não muda regras
Perfil de público	✗	Não inferido
Grade de atrações	✗	Não existe no schema

Próximos Passos

1. **Criar tabela de contexto** com parâmetros por tipo de evento
2. **Thresholds dinâmicos** — fila aceitável varia por tipo
3. **Integrar clima em tempo real** — Open-Meteo durante evento
4. **Grade de atrações** — campo no evento para horários de pico esperados
5. **Inferência de perfil** — deduzir público pelos dados de consumo

Para Desenvolvedores

Regra fundamental: Antes de qualquer decisão, a Syntropy deve perguntar: “Em que contexto esse dado existe?” Se não souber o contexto, deve reduzir a confiança da decisão (ver Confidence Engine).

Anti-pattern: Nunca usar thresholds fixos para todos os tipos de evento. O que é normal para um festival é crítico para um corporativo.

Syntropy Confidence Engine

Como a Syntropy mede a confiança nas próprias decisões.

Toda IA deveria saber o quanto confia no que está dizendo. O Confidence Engine é o mecanismo que conecta **grau de certeza** a **nível de autonomia**. Quanto mais confiante, mais autonomia. Quanto menos confiante, mais humano no loop.

O Princípio

Confiança alta

→ Executa sozinha

Confiança média

→ Sugere e espera aprovação

Confiança baixa

→ Pedir confirmação com opções

Confiança mínima

→ Não responde. Diz "não sei".

A Syntropy **nunca blefa**. Se não tem dados suficientes, ela diz. Isso é o que diferencia um sistema confiável de um sistema perigoso.

Escala de Confiança

Nível	Score	Comportamento	Exemplo
Certeza	90-100%	Executa automaticamente	“Estoque zerou → Marcar como indisponível”
Alta	75-89%	Executa e notifica depois	“Fila > 15min + causa clara → Ativa Smart Routing”
Média	50-74%	Sugere com justificativa	“Gin Tônica encalhando → Sugerir promoção?”
Baixa	25-49%	Apresenta opções	“Vendas caindo. Pode ser: A) fim natural, B) problema. O que acha?”
Incerta	0-24%	Não opina	“Dados insuficientes para recomendar”

Como a Confiança é Calculada

A confiança de uma decisão é o produto de 5 fatores:

$$\text{CONFIANÇA} = \text{Dados} \times \text{Contexto} \times \text{Histórico} \times \text{Consenso} \times \text{Reversibilidade}$$

Fator 1: Qualidade dos Dados

Situação	Multiplicador
Dados em tempo real, completos	1.0
Dados com 5-15min de atraso	0.8
Dados parciais (alguns bares sem info)	0.6
Dados estimados/inferidos	0.4
Sem dados diretos	0.2

Fator 2: Clareza do Contexto

Situação	Multiplicador
Contexto explícito (produtor configurou)	1.0
Contexto inferido com alta confiança	0.8
Contexto ambíguo	0.5
Contexto desconhecido	0.3

Fator 3: Histórico Disponível

Situação	Multiplicador
5+ eventos anteriores do produtor	1.0
2-4 eventos anteriores	0.8
1 evento anterior	0.6
Primeiro evento, mas tem benchmark	0.4
Primeiro evento, sem benchmark	0.2

Fator 4: Consenso entre Motores

Situação	Multiplicador
Rules + LLM concordam	1.0
Apenas LLM detectou	0.7
Apenas Rules detectou	0.8
Rules e LLM discordam	0.3

Fator 5: Reversibilidade da Ação

Situação	Multiplicador
Totalmente reversível (ex: notificação)	1.0
Parcialmente reversível (ex: promoção)	0.8
Difícil de reverter (ex: realocar equipe)	0.6
Irreversível (ex: cancelar pedido)	0.3

Cálculo exemplo

Situação: "Cerveja vai acabar em 45min"

Dados: 1.0 (tempo real, estoque atualizado)
Contexto: 0.8 (festival outdoor, inferido do tipo)
Histórico: 0.6 (1 evento anterior)
Consenso: 1.0 (rules + LLM concordam)
Reversível: 1.0 (alertar é reversível)

CONFIANÇA = $1.0 \times 0.8 \times 0.6 \times 1.0 \times 1.0 = 0.48 \rightarrow 48\%$

- Nível: BAIXA
- Comportamento: Apresenta opções ao produtor
- "Cerveja pode acabar em ~45min. Quer que eu alerte a equipe?"

Situação: "Estoque zerou" (fato, não previsão)

Dados: 1.0 (sistema registrou zero)
Contexto: 1.0 (irrelevante – é fato)
Histórico: 1.0 (irrelevante – é fato)
Consenso: 1.0 (não precisa de LLM)
Reversível: 1.0 (marcar indisponível é reversível)

CONFIANÇA = $1.0 \times 1.0 \times 1.0 \times 1.0 \times 1.0 = 1.0 \rightarrow 100\%$

- Nível: CERTEZA
- Comportamento: Executa automaticamente
- Marca item como indisponível no cardápio

Confiança × Autonomia (Hierarquia)

NÍVEL 0 – OBSERVAR

Confiança: Qualquer

Ação: Apenas registrar e monitorar

Exemplo: "Vendas estão 10% acima do baseline"

NÍVEL 1 – INFORMAR

Confiança: > 25%

Ação: Mostrar insight no dashboard

Exemplo: "Bar 2 está mais lento que os outros"

NÍVEL 2 – SUGERIR

Confiança: > 50%

Ação: Card com sugestão + botão aprovar/rejeitar

Exemplo: "Sugestão: Ativar promoção de Gin Tônica?"

NÍVEL 3 – EXECUTAR E NOTIFICAR

Confiança: > 75%

Ação: Faz a ação e avisa o produtor depois

Exemplo: "Smart Routing ativado (fila > 15min)"

NÍVEL 4 – EXECUTAR SILENCIOSAMENTE

Confiança: > 90% + Ação reversível + Padrão conhecido

Ação: Faz sem avisar (aparece no log)

Exemplo: "Item marcado indisponível (estoque zero)"

Calibração da Confiança

A confiança não é estática. Ela se calibra com feedback:

Feedback positivo (aumenta confiança futura)

Syntropy sugere → Produtor aprova → Resultado positivo
→ Confiança para situações similares: +5%
→ Se 5 aprovações consecutivas: Subir de nível de autonomia

Feedback negativo (reduz confiança futura)

Syntropy sugere → Produtor rejeita
→ Confiança para situações similares: -10%
→ Se 3 rejeições consecutivas: Descer de nível de autonomia

Syntropy executa → Resultado negativo
→ Confiança: -15%
→ Reverter para nível SUGERIR

Reset de confiança

Novo tipo de evento → Reset para baseline (50%)
Novo produtor → Reset para mínimo (25%)
Mudança drástica de contexto → Reduzir 30%

Transparência de Confiança

O produtor sempre pode ver o nível de confiança:

[Aprovar] [Ajustar] [Rejeitar]

Situação	Resposta
Primeiro evento + sem benchmark	“Ainda não tenho dados suficientes para prever”
Dados conflitantes sem resolução	“Estou vendo sinais contraditórios. Preciso de mais tempo”
Contexto completamente novo	“Esse tipo de evento é novo para mim. Vou observar”
Ação irreversível + baixa confiança	“Não recomendo agir agora. Risco alto”
Produtor rejeitou 3x o mesmo tipo	“Entendi que você prefere decidir isso. Não vou sugerir mais”

Implementação Atual vs Planejada

Capacidade	Status	Onde
Níveis de autonomia (observar/sugerir/executar)	✓	<code>intelligence-engine.ts</code>
Score de confiança por sugestão	✗	Não calculado
Calibração por feedback	✗	Não existe
Transparência (mostrar % ao produtor)	✗	UI não mostra
“Não sei” explícito	◆	Parcial (não gera sugestão, mas não explica)
Thresholds por nível	✗	Hardcoded em 1 nível

Próximos Passos de Implementação

1. Adicionar `confidenceScore` a cada `SuggestedAction` — calculado pelos 5 fatores
2. UI mostrar barra de confiança — produtor entende por que a Syntropy está sugerindo vs executando
3. Tabela de feedback — registrar aceite/rejeição por tipo de sugestão
4. Calibração automática — ajustar thresholds baseado em histórico de feedback
5. Nível 4 (execução silenciosa) — só ativar após 10+ aprovações do mesmo tipo

Para Desenvolvedores

Regra fundamental: A Syntropy nunca deve agir com confiança que não tem. Se o score é baixo, ela reduz a autonomia. Se é alto, ela age. Mas nunca blefa.

Anti-pattern: Nunca usar confiança fixa (ex: “todas as sugestões têm 80%”). A confiança deve ser calculada dinamicamente com base nos dados reais disponíveis.

Syntropy Evolution

Como a Syntropy amadurece de assistente a operador autônomo.

Este documento descreve a jornada de maturidade da Syntropy ao longo de 5 anos. Não é uma lista de features — é a evolução de uma inteligência que acumula experiência operacional e ganha autonomia progressiva.

A Jornada



2026 — Assistente

“Eu observo, analiso e sugiro. Você decide.”

Capacidades

O que faz	Como faz	Autonomia
Planeja evento	Conversa no onboarding	Sugere, produtor aprova
Monitora em tempo real	Dashboard + alertas	Observa e informa
Detecta anomalias	Rules + LLM	Alerta o produtor
Sugere ações	Cards com impacto estimado	Produtor executa
Gera relatório pós-evento	Vision automático	Entrega sem pedir
Distribui pedidos	Smart Routing	Executa (reversível)

Limitações

- Não executa ações de impacto sem aprovação
- Não aprende entre produtores (dados isolados)
- Não prevê com base em clima
- Não sugere promoções proativamente
- Precisa de dados do evento atual (sem memória de longo prazo)

Métrica de sucesso

“O produtor diz: a Syntropy me ajudou a ver coisas que eu não via.”

2027 — Especialista

“Eu já vi isso antes. Sei o que funciona.”

Novas capacidades

O que ganha	Diferença vs 2026	Pré-requisito
Memória de produtor	Lembra eventos anteriores	Memory Engine
Previsão calibrada	Usa histórico real, não genérico	3+ eventos/produtor
Promoções automáticas	Sugere E executa (com aprovação)	Confidence Engine
Contexto dinâmico	Ajusta em tempo real (clima, horário)	Context Engine
Benchmark	Compara com produtores similares	50+ produtores na rede
Onboarding inteligente	Pré-preenche baseado no perfil	Producer Profiles

O que muda na prática

ANTES (2026):

Produtor: "Quanto de cerveja compro?"

Syntropy: "Baseado em eventos similares, sugiro 500 unidades."

DEPOIS (2027):

Syntropy: "Nos seus últimos 3 festivais, o consumo médio foi 480. Mas este é outdoor e está previsto 34°C. Recomendo 580 unidades (+20% por calor). Nos últimos eventos com calor similar, o consumo subiu 22%."

Métrica de sucesso

"O produtor diz: a Syntropy me conhece. Ela sabe como meu evento funciona."

2028 — Coordenadora

“Eu organizo a operação inteira. Você só intervém quando quer.”

Novas capacidades

O que ganha	Diferença vs 2027	Pré-requisito
Execução autônoma (nível 3)	Age e notifica depois	Confiança > 75% calibrada
Coordenação multi-bar	Rebalanceia equipe automaticamente	Dados de performance
Precificação dinâmica	Ajusta preços por demanda	Elasticidade calculada
Comunicação com participante	Push personalizado	CRM ativo
Previsão de receita	Projeta faturamento antes do evento	Modelo treinado
Sugestão de cardápio	Monta cardápio ideal	Dados de rede

O que muda na prática

ANTES (2027):

Syntropy: "Bar 2 está lento. Sugestão: realocar 1 operador."

Produtor: [Aprovar]

DEPOIS (2028):

Syntropy: [Já realocou]

Notificação: "Realoquei 1 operador para o Bar 2.
Fila caiu de 12min para 5min. Impacto: +R\$ 1.800/h."

Métrica de sucesso

“O produtor diz: eu nem preciso ficar olhando. A Syntropy resolve.”

2029 — Supervisora

“Eu cuido de tudo. Você só intervém se quiser mudar a estratégia.”

Novas capacidades

O que ganha	Diferença vs 2028	Pré-requisito
Execução autônoma (nível 4)	Age silenciosamente	10+ aprovações por tipo
Planejamento completo	Monta evento sozinha	Perfil maduro
Gestão de fornecedores	Sugere compras, negocia	Marketplace integrado
Previsão de lucro	Projeta P&L antes do evento	Modelo financeiro
Multi-evento	Coordena vários eventos simultâneos	Infraestrutura escalada
Aprendizado de rede	Transfere conhecimento entre produtores	500+ produtores

O que muda na prática

ANTES (2028):

Produtor: "Preciso montar o próximo evento."

Syntropy: "Baseado no seu perfil, sugiro: [plano completo]"

DEPOIS (2029):

Syntropy: "Seu próximo evento é dia 15. Já montei o plano:

- Cardápio otimizado (baseado nos últimos 8 eventos)
- Estoque calculado (fornecedor X tem melhor preço)
- Equipe dimensionada (12 operadores, escala por horário)
- Receita projetada: R\$ 145.000 (±8%)
- Risco principal: chuva (30% chance) → plano B pronto

Quer revisar ou posso seguir?"

Métrica de sucesso

“O produtor diz: a Syntropy é minha sócia operacional. Ela pensa no evento antes de mim.”

2030 — Operadora Autônoma

“Eu opero o evento inteiro. Você aparece para curtir.”

Novas capacidades

O que ganha	Diferença vs 2029	Pré-requisito
Operação end-to-end	Do planejamento ao relatório, sem intervenção	Confiança máxima
Negociação automática	Fecha com fornecedores	Marketplace maduro
Otimização financeira	Maximiza lucro em tempo real	Modelo econômico
Predição de mercado	Sugere novos tipos de evento	Dados de rede massivos
White-label	Syntropy como serviço para outras plataformas	API aberta
Auto-evolução	Identifica e implementa melhorias	Meta-learning

O que muda na prática

ANTES (2029):
Produtor monta evento com ajuda da Syntropy.

DEPOIS (2030):
Syntropy: "Identifiquei oportunidade: festival de cerveja artesanal no bairro X tem demanda reprimida (dados de busca + eventos próximos lotaram). Projeção: R\$ 200k faturamento, 65% margem

Quer que eu monte o evento completo?

- Local: já identifiquei 3 opções
- Fornecedores: cotações prontas
- Equipe: banco de operadores disponível
- Marketing: audiência mapeada

Investimento estimado: R\$ 45k. ROI projetado: 3.4x."

Métrica de sucesso

“O produtor diz: a Syntropy não é minha ferramenta. É minha parceira de negócio.”

Gatilhos de Evolução

A Syntropy não avança de nível por tempo — avança por **conquista de confiança**:

De → Para	Gatilho	Evidência
Assistente → Especialista	3+ eventos do produtor	Memória suficiente
Especialista → Coordenadora	80%+ sugestões aceitas	Confiança calibrada
Coordenadora → Supervisora	50+ execuções sem reversão	Autonomia comprovada
Supervisora → Operadora	Produtor ativa “modo autônomo”	Decisão explícita

Regra de segurança

A Syntropy NUNCA sobe de nível sem evidência.

A Syntropy SEMPRE pode descer de nível se errar.

O produtor SEMPRE pode forçar qualquer nível.

Evolução por Produtor (Não Global)

A Syntropy evolui **por produtor**, não globalmente:

Produtor A (10 eventos): Nível COORDENADORA

Produtor B (2 eventos): Nível ASSISTENTE

Produtor C (30 eventos): Nível SUPERVISORA

Cada produtor tem sua própria jornada. A Syntropy respeita o ritmo de cada um.

Implicações para Investidores

Ano	Valor para o produtor	Moat
2026	Visibilidade (vê o que não via)	Dados começam a acumular
2027	Previsibilidade (sabe o que vai acontecer)	Histórico é insubstituível
2028	Eficiência (opera melhor com menos esforço)	Calibração é única
2029	Estratégia (planeja melhor que qualquer humano)	Inteligência proprietária
2030	Autonomia (o evento roda sozinho)	Lock-in total por valor

O flywheel de evolução

Mais eventos → Mais dados → Mais experiência operacional
→ Melhor previsão → Mais confiança → Mais autonomia
→ Menos trabalho para produtor → Mais eventos
→ (ciclo se repete, acelerando)

A Frase que Resume

“A Syntropy não é programada para ficar melhor. Ela acumula experiência operacional que a torna insubstituível.”

Isso é o que nenhum concorrente pode copiar: não é código, é **tempo × dados × contexto × confiança**.